



Universidad Tecnológica de Querétaro

Ingeniería en Desarrollo y Gestión de Software

Seguridad en Desarrollo de Software

SEGG-U1-P1 — OWASP Top 10: Investigación y análisis de vulnerabilidades

Profesor: Gerardo Ramírez Villarreal

Grupo: IDGS17

Alumno: Aldair Hernández Cruz

A01:2025 Control de acceso averiado	
Descripción	La vulnerabilidad top 1, que evidentemente es la más común en los sitios web, se da en la parte del control de accesos (login, registro), básicamente a través de url's modificadas, configuración incorrecta de cors, APIs fácilmente accesibles por falta de filtros, etc.
Ejemplo de ataque	Un atacante puede modificar la url y tener acceso a la cuenta de una persona, un ejemplo, podría colocar un id(numero) en acct y logearse; https://example.com/app/accountInfo?acct=notmyacct O una mala gestión en los permisos o privilegios de las distintas rutas (paginas o vistas) de nuestro sitio, pueden quedar expuestas fácilmente.
Impacto potencial	El impacto de romper el control de acceso suele ser crítico y de alto riesgo. Entre las consecuencias más graves se encuentran: exposición de los datos almacenados de los usuarios, manipulación de los mismos, manipulación de privilegios dentro del sitio y tomar control de el.
Método de mitigación	Para mitigar esta principal vulnerabilidad, seria a través de una buena gestión de los permisos y privilegios de las rutas internas del sistema, validaciones desde el backend no mediante url, implementación de middlewares.
Caso real 1	Filtración de datos de First American Financial Corp (2019) El equipo interno de la empresa descubrió una vulnerabilidad de Referencia Directa a Objetos Insegura (IDOR) durante una prueba de penetración manual. Este fallo permitía a los usuarios acceder a información privada mediante una URL específica, modificando secuencialmente sus números. Sin la autenticación adecuada, cualquier usuario podía acceder libremente a cualquier información.
Caso real 2	El bug de las salas de Zoom (2020) Los enlaces de las reuniones dependían únicamente de un ID numérico corto incluido en la URL. El backend no validaba de forma estricta si el usuario que intentaba unirse tenía permiso o si venía de una invitación legítima, permitiendo que scripts automatizados adivinaran los IDs.

Ejemplificación de la vulnerabilidad en código:

A01:2025 Control de acceso averiado

Este fallo ocurre cuando el backend confía ciegamente en el ID que envía el cliente en la URL, en lugar de validar si ese usuario es realmente el dueño de los datos.

Código Vulnerable

El servidor toma el id de la URL directamente y busca la información sin verificar la sesión.

```
// El atacante puede cambiar el id en la URL para ver datos de otros
app.get('/api/v1/usuarios/:id/perfil', async (req, res) => {
  const usuario = await baseDatos.buscarPorId(req.params.id);
  res.json(usuario);
});
```

Código Seguro

Se lee el ID del usuario **directamente desde el token o sesión activa** del servidor (previamente validada por un *middleware* de autenticación).

```
// El ID se extrae de la sesión segura del servidor, no de la URL
app.get('/api/v1/usuarios/perfil', usuarioAutenticado, async (req, res) => {
  const usuario = await baseDatos.buscarPorId(req.user.id);
  res.json(usuario);
});
```

A02:2025 Configuración de seguridad incorrecta	
Descripción	Esta vulnerabilidad se trata de la falta de una buena configuración del lado del servidor, como puertos, rutas, cuentas (usuarios root), exposición excesiva de errores (logs específicos), etc.
Ejemplo de ataque	Un usuario accede a las herramientas de desarrollo de chrome y encuentra en la consola errores muy específicos con información sensible que pueden facilitar el acceso al servidor o filtrar datos de usuarios.
Impacto potencial	Impacto severo, ya que si encuentran o burlan esta vulnerabilidad se puede perder el control de nuestro propio servidor, daños a nuestro sistema, etc.
Método de mitigación	Una arquitectura de aplicación segmentada proporciona una separación eficaz y segura, implementar un método de capturar todos los mensajes de error excesivos.
Caso real 1	La filtración del código fuente de Twitch (2021) Un usuario anónimo publicó un enlace de descarga con 125 GB de datos confidenciales de Twitch, que incluían todo el código fuente de la plataforma, y los pagos detallados de los <i>streamers</i>. Los datos fueron exfiltrados debido a una mala configuración en un servidor proxy inverso (como Nginx o balanceadores de carga) permitió que un recurso de almacenamiento interno, que debía ser estrictamente privado, quedara accesible de forma pública a través de internet sin pasar por los filtros de autenticación.
Caso real 2	El hackeo masivo a Yahoo! (2014) Un grupo de actores respaldados por un estado logró comprometer los servidores de Yahoo!, robando la información de cuentas de más de 500 millones de usuarios (nombres, correos, hashes de contraseñas). Al enviar peticiones maliciosas (con caracteres extraños) a ciertas URLs, el servidor no manejó los errores de forma genérica. En su lugar, devolvió una pantalla de error del framework de desarrollo que incluía el "Stack Trace" (la traza del código fuente) y los detalles exactos de la consulta SQL que falló.

A03:2025 Fallos en la cadena de suministro de software	
Descripción	Los fallos en la cadena de suministro de software son interrupciones u otras deficiencias en el proceso de creación, distribución o actualización de software. Como el no llevar un control de versiones, no contar con soporte.
Ejemplo de ataque	El atacante inyecta código malicioso en los scripts automatizados de instalación del paquete. Cuando otros desarrolladores o servidores descargan la librería actualizándola de forma rutinaria, el código malicioso se ejecuta en sus entornos locales o servidores de producción de manera silenciosa.
Impacto potencial	Pérdida de confianza e integridad: Si tu software distribuido (como una app móvil o web) se compila con una dependencia infectada, tus clientes finales se verán comprometidos, destruyendo la reputación de tu marca.
Método de mitigación	Eliminación de dependencias o bibliotecas obsoletas o innecesarias, realizar gestión de versiones del sistema, actualización de las herramientas de trabajo, así como el entorno de desarrollo.
Caso real 1	Ataque a node.js (2025) El Shai-Huludataque a la cadena de suministro en 2025 fue el primer gusano npm autopropagante exitoso. Los ataques introdujeron versiones maliciosas de paquetes populares, que utilizaban un script posterior a la instalación para recopilar y extraer datos confidenciales a repositorios públicos de GitHub.
Caso real 2	El ataque por "Confusión de Dependencias" a PyTorch (2022) la biblioteca de código abierto PyTorch (una de las herramientas de Machine Learning más utilizadas del mundo, desarrollada originalmente por Meta) sufrió un ataque a su cadena de suministro que afectó a miles de desarrolladores. PyTorch utiliza un paquete interno llamado torchtriton. Este paquete no se alojaba en el repositorio público oficial de Python (PyPI), sino que se descargaba desde un servidor privado de los desarrolladores de PyTorch. Un atacante se dio cuenta de esto, fue al registro público de PyPI y registró un paquete malicioso exactamente con el mismo nombre: torchtriton. El atacante subió una versión con un número deliberadamente más alto que la versión que usaba PyTorch internamente. <i>pip</i> priorizó la instalación de forma automática de la última versión. Los desarrolladores terminaron descargando e instalando el paquete del atacante. El código malicioso contenía un troyano.

A04:2025 Fallos criptográficos

Descripción	En términos generales, todos los datos en tránsito deben cifrarse en la capa de transporte (capa 4 del modelo OSI). Más allá de asegurar la capa de transporte, es importante determinar qué datos necesitan cifrado en reposo y qué datos necesitan cifrado adicional en tránsito (en la capa de aplicación , capa 7 del modelo OSI).
Ejemplo de ataque	La base de datos de contraseñas utiliza hashes simples para almacenar las contraseñas de todos los usuarios. Una vulnerabilidad en la carga de archivos permite a un atacante acceder a la base de datos de contraseñas.
Impacto potencial	Pérdida total de confidencialidad: Acceso no autorizado a secretos del sistema, credenciales de administrador, datos financieros (tarjetas de crédito) y datos de salud o personales (PII). Multas regulatorias catastróficas: Incumplimiento grave de normativas internacionales de seguridad de datos como PCI-DSS (para pagos con tarjeta) o leyes locales de protección de datos personales, lo que conlleva demandas millonarias.
Método de mitigación	Clasificar y etiquetar los datos procesados, almacenados o transmitidos por una aplicación. Identificar qué datos son sensibles según las leyes de privacidad, los requisitos reglamentarios o las necesidades del negocio. Desactive el almacenamiento en caché para las respuestas que contengan datos confidenciales.
Caso real 1	La vulnerabilidad de interceptación en las apps de Nissan Leaf (2016) La aplicación móvil del coche se comunicaba con los servidores de Nissan utilizando el protocolo HTTP sin cifrar (en lugar de HTTPS de forma estricta), y la API de control no cifraba ni autenticaba las peticiones mediante tokens seguros. La API solo dependía del número de chasis (VIN) del vehículo transmitido en texto plano en la URL.
Caso real 2	Filtración de datos de Facebook (2019) La filtración de datos se produjo porque Facebook almacenaba las contraseñas de los usuarios en texto plano, es decir, sin cifrar. Esto facilitó que los hackers accedieran a las cuentas de los usuarios adivinando o descifrando las contraseñas.

A05:2025 Inyección	
Descripción	La vulnerabilidad de inyección ocurre cuando datos no confiables introducidos por el usuario son interpretados inadvertidamente como parte de un comando o consulta. El intérprete ejecuta estos datos maliciosos, lo que altera el flujo lógico planeado de la aplicación.
Ejemplo de ataque	Imagina un formulario de login que valida las credenciales construyendo una consulta SQL mediante concatenación directa de strings en el backend (Node.js/Express).
Impacto potencial	Perdida de confidencialidad, destrucción o alteración de datos y ejecución remota de código.
Método de mitigación	Validación de entradas (Lista blanca): Validar que los datos tengan el tipo, longitud y formato esperado (por ejemplo, usando librerías como <i>Zod</i> o <i>Express-validator</i>).
Caso real 1	El hackeo masivo a Sony Pictures (2011) Los atacantes utilizaron técnicas de Inyección SQL automatizada contra los formularios web del sitio. Al no estar parametrizadas las consultas del backend, pudieron "inyectar" comandos que forzaron a la base de datos de Sony a volcar todo su contenido, robando cupones de música, contraseñas, correos electrónicos y datos personales de más de un millón de usuarios registrados.
Caso real 2	La filtración de datos de la infraestructura de Equifax (2017) El ataque se produjo a través de una vulnerabilidad de inyección conocida en el framework Apache Struts (CVE-2017-5638). El sistema permitía una Inyección de Comandos a través de las cabeceras HTTP (Content-Type) . Al enviar código malicioso modificado en la cabecera, el framework lo interpretaba incorrectamente y lo ejecutaba directamente en el sistema operativo del servidor.

Ejemplificación de la vulnerabilidad en código:

A05:2025 Inyección

Ocurre cuando se concatenan directamente variables del usuario dentro de una cadena de texto que se enviará a la base de datos, permitiendo que el texto sea interpretado como comandos.

Código Vulnerable

Al usar comillas y concatenar el string, el atacante puede romper la estructura de la consulta.

```
// Si el usuario escribe: ' OR '1'='1 , se salta el login
app.post('/api/v1/login', async (req, res) => {
  const query = `SELECT * FROM usuarios WHERE email = '${req.body.email}'`;
  const resultado = await baseDatos.ejecutarQueryDirecta(query);
  res.json(resultado);
});
```

Código Seguro

Se utilizan **consultas preparadas (parámetros)**. El motor de la base de datos compila la estructura primero y trata el input del usuario estrictamente como texto plano.

```
// El símbolo ? actúa como marcador de posición seguro
app.post('/api/v1/login', async (req, res) => {
  const query = 'SELECT * FROM usuarios WHERE email = ?';
  const resultado = await baseDatos.ejecutar(query, [req.body.email]);
  res.json(resultado);
});
```


A06:2025 Diseño inseguro	
Descripción	El diseño inseguro se refiere a los defectos en la arquitectura y en el diseño del software. Representa una falta de perfiles de seguridad automatizados, modelado de amenazas o principios de diseño sólido
Ejemplo de ataque	Un flujo de trabajo de recuperación de credenciales podría incluir preguntas y respuestas, lo cual está prohibido por NIST 800-63b, OWASP ASVS y OWASP Top 10. Las preguntas y respuestas no son confiables como prueba de identidad, ya que más de una persona puede conocer las respuestas. Esta funcionalidad debe eliminarse y reemplazarse con un diseño más seguro.
Impacto potencial	Abuso legítimo de funciones (Logic Flaws): Los atacantes pueden explotar el flujo lógico para obtener beneficios económicos, saltarse pasarelas de pago, o adquirir productos gratis simplemente siguiendo los pasos que el sistema permite.
Método de mitigación	Ciclo de Vida de Desarrollo Seguro (SDLC): Utilizar metodologías de desarrollo que incluyan revisiones de arquitectura, diseño de casos de abuso y pruebas de lógica de negocio en cada sprint.
Caso real 1	El exploit del flujo de créditos en Zoom (2020) El sistema estaba diseñado para regalar créditos o tiempo premium a usuarios nuevos que se registraran con correos institucionales o de empresas. Sin embargo, el diseño del backend solo validaba que el string del correo terminara con un dominio específico (ejemplo: @universidad.edu), pero no requería una verificación real por correo electrónico (hacer clic en un enlace de activación) antes de otorgar el beneficio premium.
Caso real 2	La vulnerabilidad de omisión de pago en Etsy (2014) La plataforma permitía a los usuarios pagar utilizando múltiples divisas. El diseño del sistema delegaba la conversión de monedas al navegador del cliente y luego enviaba el total al backend. Si un atacante modificaba el flujo lógico, añadía un producto de \$1,000 USD, pero cambiaba el identificador de la divisa local a una moneda de bajísimo valor (como el Dong vietnamita o monedas de juegos que equivalen a fracciones de centavo) justo antes de enviar la orden, el sistema procesaba el pago. El código no tenía inyecciones ni errores de servidor; simplemente el diseño confiaba ciegamente en el flujo de datos que el cliente manipulaba.

A07:2025 Fallos de autenticación	
Descripción	Cuando un atacante logra engañar a un sistema para que reconozca a un usuario inválido o incorrecto como legítimo, existe esta vulnerabilidad.
Ejemplo de ataque	Los tiempos de espera de las sesiones de las aplicaciones no se implementan correctamente. Un usuario utiliza un ordenador público para acceder a una aplicación y, en lugar de seleccionar "cerrar sesión", simplemente cierra la pestaña del navegador.
Impacto potencial	Secuestro de sesiones (Session Hijacking), Toma de control de cuentas a gran escala (Account Takeover - ATO): Mediante ataques automatizados de fuerza bruta o <i>Credential Stuffing</i> (probar combinaciones de correos y contraseñas filtradas),
Método de mitigación	Siempre que sea posible, implemente y haga cumplir el uso de la autenticación multifactor para prevenir el relleno automatizado de credenciales, los ataques de fuerza bruta y los ataques de reutilización de credenciales robadas, Implemente comprobaciones de contraseñas débiles
Caso real 1	El ataque de Credential Stuffing a Disney+ (2019) La plataforma no contaba en sus inicios con sistemas robustos de limitación de tasa de peticiones (<i>Rate Limiting</i>) en el endpoint de inicio de sesión, ni exigía autenticación multifactor (MFA). Esto permitió que bots automatizados atacaran el formulario de login probando millones de credenciales que habían sido filtradas previamente en otras brechas de seguridad de internet
Caso real 2	El robo de tokens de sesión en Slack (2022) Un atacante logró infiltrarse en los canales internos de comunicación de la empresa de videojuegos EA (Electronic Arts) a través de su plataforma de Slack, lo que derivó en el robo de 750 GB de datos, incluido el código fuente de juegos como <i>FIFA 21</i> . El atacante no adivinó la contraseña del empleado, sino que compró una <i>cookie de sesión</i> (token de autenticación) activa en un mercado de malware en internet (la cual fue extraída previamente de la computadora del empleado mediante un troyano). Como el sistema de autenticación de Slack aceptaba la cookie clonada sin validar si el navegador, la IP o el comportamiento del usuario habían cambiado drásticamente, le permitió el acceso directo saltándose el inicio de sesión tradicional.

A08:2025 Fallos en la integridad del software o de los datos	
Descripción	Los fallos de integridad del software y los datos se relacionan con código e infraestructura que no protegen contra el uso de código o datos no válidos o no confiables, que a menudo se tratan como si fueran confiables y válidos.
Ejemplo de ataque	Uso de un paquete de una fuente no confiable: Un desarrollador tiene dificultades para encontrar la versión actualizada de un paquete que necesita, por lo que lo descarga de un sitio web en lugar del gestor de paquetes habitual y confiable. El paquete no está firmado, por lo que no hay forma de garantizar su integridad. El paquete contiene código malicioso.
Impacto potencial	Ejecución Remota de Código (RCE), Actualizaciones maliciosas (Firmware/Software Hijacking), Alteración del flujo lógico de la aplicación, etc
Método de mitigación	Asegúrese de que las bibliotecas y dependencias, como npm o Maven, solo consuman repositorios de confianza, utilice firmas digitales o mecanismos similares para verificar que el software o los datos provienen de la fuente esperada
Caso real 1	El ataque de inyección de firmware a ASUS (Operation ShadowHammer - 2019) Los servidores de actualización oficiales de ASUS fueron comprometidos, lo que permitió a un grupo de atacantes distribuir una actualización de software maliciosa a más de un millón de computadoras, Aunque la herramienta descargaba la actualización desde los servidores correctos, el sistema de los usuarios aceptaba el archivo basándose únicamente en el nombre o la ruta sin realizar una validación estricta de la integridad del hash y los certificados digitales específicos del nuevo código inyectado. Los atacantes lograron utilizar un certificado digital válido de la propia empresa que no había sido revocado para firmar el malware.
Caso real 2	El hackeo masivo a CCleaner (2017) Un grupo de atacantes avanzados logró infiltrarse en la red interna de desarrollo de Piriform meses antes del lanzamiento. Modificaron directamente el código fuente antes de que el programa fuera compilado y empaquetado. El gran fallo de integridad ocurrió porque el pipeline automatizado de la empresa firmó digitalmente el software malicioso con el certificado oficial y legítimo de la compañía sin realizar comprobaciones ni auditorías cruzadas de la integridad del binario final contra el repositorio de código limpio. El malware venía dormido y, tras unos días de instalarse, se activaba para recopilar nombres de computadoras, software instalado y direcciones IP, enviando todo a un servidor controlado por los atacantes.

A09:2025 Fallos en el registro y las alertas de seguridad	
Descripción	La falta de registros (logs) y un monitoreo deficiente impiden que la empresa se entere de que está siendo atacada en tiempo real. Los eventos auditable, como los inicios de sesión, los intentos fallidos de inicio de sesión y las transacciones de alto valor, no se registran o se registran de forma inconsistente (por ejemplo, solo se registran los inicios de sesión exitosos, pero no los intentos fallidos).
Ejemplo de ataque	Una aerolínea india sufrió una filtración de datos que afectó a más de diez años de información personal de millones de pasajeros, incluyendo datos de pasaportes y tarjetas de crédito. La filtración se produjo en un proveedor externo de alojamiento en la nube, quien notificó a la aerolínea tiempo después.
Impacto potencial	Infecciones persistentes y silenciosas, Imposibilidad de análisis forense: Cuando finalmente se descubre la brecha, la falta de históricos de logs hace imposible saber qué datos se robaron, cómo entró el atacante, qué cuentas se vieron afectadas o cuándo empezó el ataque. Incumplimiento de normativas: La mayoría de las leyes de privacidad exigen reportar las brechas de seguridad en un plazo menor a 72 horas desde que ocurren
Método de mitigación	Asegúrese de que todos los fallos de inicio de sesión, control de acceso y validación de entrada del lado del servidor se registren con suficiente contexto de usuario para identificar cuentas sospechosas o maliciosas y se conserven durante el tiempo suficiente para permitir un análisis forense diferido. Asegúrese de que se registre cada parte de su aplicación que contenga un control de seguridad, tanto si tiene éxito como si falla.
Caso real 1	El ataque silencioso a la Oficina de Administración de Personal de EE. UU. (OPM) (2014) Los atacantes clonaron las credenciales de un contratista legítimo para ingresar al sistema. Al no existir alertas de seguridad que auditaran accesos desde ubicaciones anómalas o en horarios extraños, el sistema trató las peticiones de descarga masiva de datos como "actividad normal". La agencia descubrió el ataque meses después, únicamente porque instalaron una nueva herramienta de seguridad externa que escaneó el tráfico del servidor de forma retroactiva.
Caso real 2	El hackeo a Uber y la extorsión oculta (2016) Los atacantes encontraron credenciales de acceso de los ingenieros de Uber expuestas en un repositorio privado de GitHub. Utilizando esas claves, accedieron directamente a sus servidores de almacenamiento en AWS (Amazon Web Services). El gran fallo fue que Uber no tenía configuradas alertas de auditoría sobre el uso de esas credenciales para descargas masivas de datos, ni logs detallados que permitieran al equipo de seguridad notar el movimiento de gigaBytes de información hacia IPs externas desconocidas.

A10:2025 Gestión inadecuada de situaciones excepcionales	
Descripción	Esta vulnerabilidad ocurre cuando la aplicación web interactúa con recursos externos (APIs, bases de datos o URLs proporcionadas por el usuario) y no gestiona adecuadamente los escenarios fuera de lo común, como caídas de servicio, respuestas inesperadas o redirecciones maliciosas. El caso más crítico es el SSRF , donde un atacante induce al servidor backend a realizar una petición HTTP maliciosa hacia una URL elegida por el atacante,
Ejemplo de ataque	Imagina que tu aplicación backend (Node.js) tiene una función que permite a los usuarios importar una foto de perfil desde una URL externa. Un atacante, en lugar de mandar una imagen pública, envía una dirección de la red interna del servidor o metadatos de la nube. El servidor procesa la petición de forma "excepcional", ejecuta el comando interno confiando en que la petición viene de sí mismo, y le regresa la información confidencial al atacante.
Impacto potencial	Acceso a redes internas protegidas, Exfiltración de credenciales de infraestructura, Denegación de Servicio(DoS)
Método de mitigación	Segmentación de red (Firewalls internos): Configurar el servidor web (o el VPS) para que tenga bloqueado por defecto el acceso hacia servicios internos no esenciales. Lista blanca de dominios (Allowlist): Si la aplicación debe consumir recursos externos, validar en el backend que la URL proporcionada pertenezca estrictamente a una lista de dominios aprobados y confiables. Validación y sanitización de URLs,
Caso real 1	La vulnerabilidad SSRF en los servidores de Microsoft Exchange (2021) El núcleo del ataque inicial se debió a un fallo de gestión en las peticiones del lado del servidor (SSRF). Los atacantes enviaban una solicitud HTTP maliciosa al puerto público del servidor de correo. El backend de Exchange procesaba la petición y, debido a un manejo inadecuado de las rutas, retransmitía la solicitud internamente a otro componente del servidor encargado de la configuración, creyendo que la petición provenía de un administrador autenticado.
Caso real 2	El SSRF en los webhooks de GitLab (2021) Un investigador de ciberseguridad descubrió una vulnerabilidad crítica (CVE-2021-22214) en las versiones comunitarias y empresariales de GitLab que permitía a usuarios no autenticados forzar al servidor a realizar peticiones internas de forma arbitraria.

Ejemplificación de la vulnerabilidad en código:

A10:2025 - Gestión inadecuada de situaciones excepcionales (SSRF)

Este error ocurre cuando el servidor realiza una petición HTTP hacia una URL que el usuario proporciona, sin validar si esa dirección apunta a un recurso privado de nuestra propia red.

Código Vulnerable

El servidor hace un fetch a cualquier dirección que le manden, pudiendo atacar su propia infraestructura interna.

```
// El atacante puede mandar: http://localhost:8080/admin/borrar-todo
app.post('/api/v1/cargar-avataar', async (req, res) => {
  const respuesta = await fetch(req.body.urlImagen);
  res.send(respuesta.body);
});
```

Código Seguro

Se valida que la URL pertenezca estrictamente a un dominio de confianza antes de realizar cualquier petición en el backend.

```
// Solo se permiten imágenes de nuestro servidor de confianza (lista blanca)
app.post('/api/v1/cargar-avataar', async (req, res) => {
  if (!req.body.urlImagen.startsWith('https://imagenes-seguras.com/')) {
    return res.status(400).send('URL no permitida');
  }
  const respuesta = await fetch(req.body.urlImagen);
  res.send(respuesta.body);
});
```

Ranking Personal:

Rank	Nombre	Justificación
1	A02:2025 Configuración de seguridad incorrecta	Considero que sea top1 porque es algo fundamental para un sistema seguro, cerrar internamente las puertas a los atacantes.
2	A08:2025 Fallos en la integridad del software o de los datos	Debemos cuidar mucho la parte de que bibliotecas, dependencias, paquetes, usamos y verificar que sean versiones limpias y oficiales.
3	A06:2025 Diseño inseguro	Es completa responsabilidad de nosotros como desarrolladores crear una buena arquitectura del proyecto para tener una buena gestión sobre la seguridad
4	A04:2025 Fallos criptográficos	fundamental sustituir los algoritmos obsoletos cuya resistencia ha sido superada por el avance en la capacidad de cómputo y el criptoanálisis por versiones estables y estandarizadas de los métodos de cifrado actuales
5	A09:2025 Fallos en el registro y las alertas de seguridad	Siempre nos percatamos de las alertas y fallos cuando es algo evidente y claro, pero nos olvidamos que pasaría ante fallos que son invisibles y que no los detecte el sistema por defecto.

Conclusión final:

La lista de Top 10 vulnerabilidades que proporciona OWASP es verdaderamente relevante para los desarrolladores y empresas de software, tomando en cuenta esta lista pueden establecer los requerimientos para elaborar un software lo mas seguro posible, o si no seguro, pero si tener la capacidad detectar las amenazas en tiempo real o en el menor tiempo posible para evitar fugas de información o alteraciones en el sistema, sabiendo que esto puede contribuir a importantes pérdidas económicas. Es una lista que nos muestra los puntos donde regularmente como humanos nos olvidamos o pasamos por alto, que tal vez muchas veces no recordamos que existe ese punto con vulnerabilidad y que usan los atacantes para crear daños. Es una lista que nos recuerda perfectamente las áreas que debemos tener cuidado, y donde debemos aplicar seguridad. En el recuento de dicha lista ya vimos que un fallo mínimo puede contribuir a un daño enorme incluso irreparable.

Personalmente la manera en que yo aplicaría esta lista seria, al momento del levantamiento de requerimientos para el desarrollo de un sistema, para definir las áreas que debemos asegurarnos blindar con una seguridad robusta y prevenir daños, y también en el punto donde el software esté listo, realizar como este checklist, e ir verificando la seguridad en los puntos que define la lista generada por OWASP.